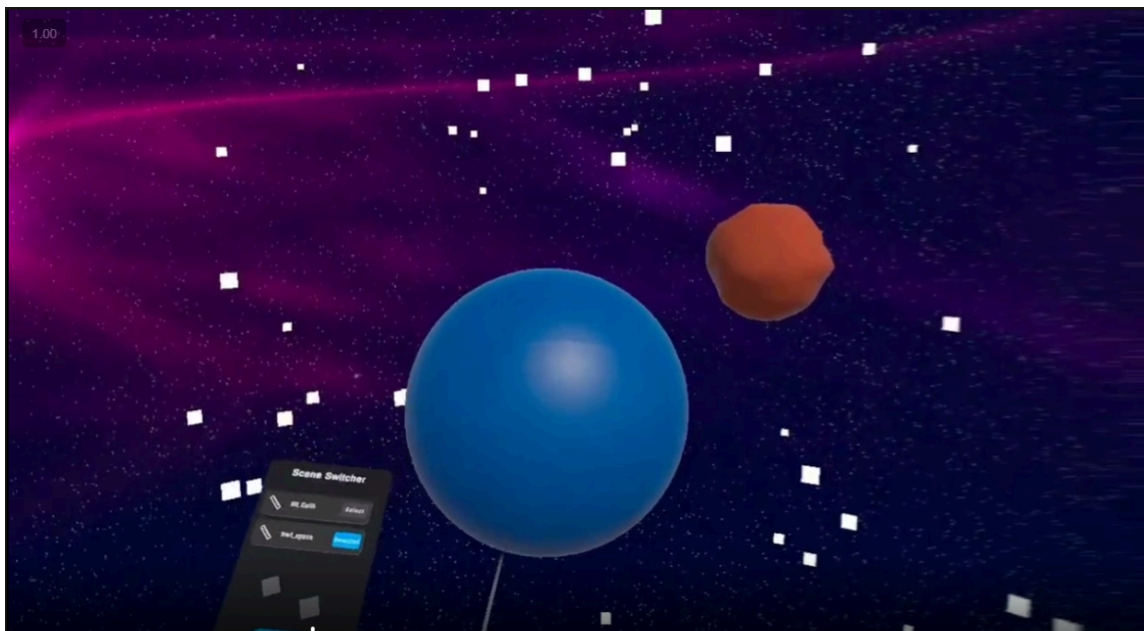
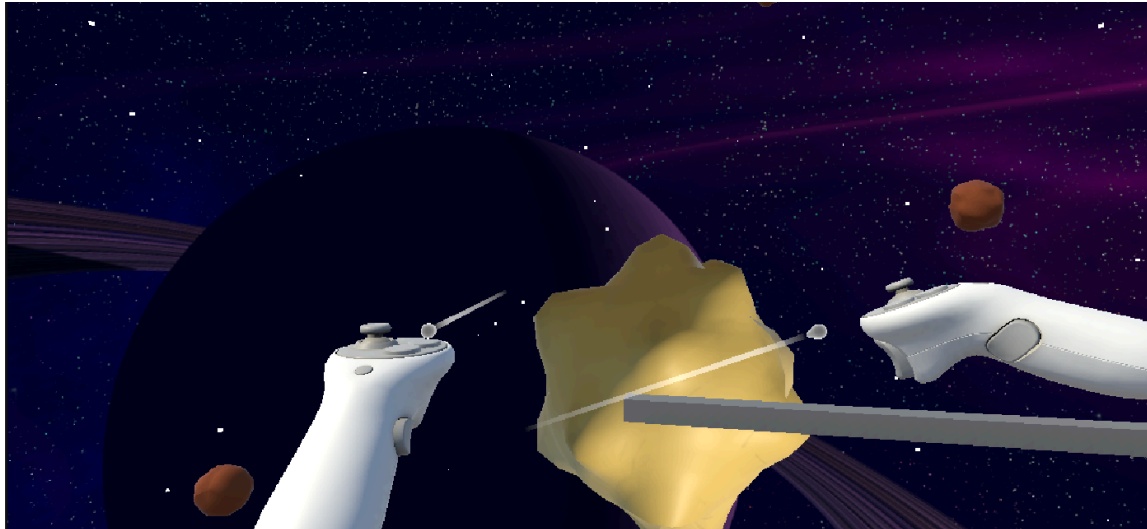




The goal of this assignment was to learn about mesh deformations. We started out by learning the fundamentals of mesh and how the world is constructed and then were tasked with applying those concepts to create 3 deformations. One that was continuous, one that reacted to input, and a third of our choosing. For my project I decided to create a space scene where the central orb is reminiscent of the earth, going from a water planet to having continents (blue -> green) when clicked and reversing if clicked with the other hand. For the continuous deformation I created a sun/moon that goes from yellow with spikes (like how a kid would draw it) to just a gray sphere for the moon. Lastly, I made asteroids that continuously float through the air which I made by deforming a sphere.

2. Technical Overview

Sun and Moon:

The main system is a 3D object that continuously deforms between a sun shape and a basic sphere shape while orbiting around a specified point and changing colors. The object is created through code using a generated sphere mesh.

At startup, the script generates the sphere's vertices and triangles using a procedural mesh. These vertices are then used as a base to create another set of vertex positions for the sun. The sun shape is created by extending vertices outward in selected directions to form pointed spikes.

The deformation is continuous and time-based. Each frame, a sine function converts the current time into a value between 0 and 1. This value is used to move each vertex between the base sphere position and the sun position. The mesh's vertices and normals are updated so that it actually changes shape over time. The material color also changes between the moon and sun colors at the same time as the vertices are changing.

The script also uses a procedural distribution of points on a sphere to determine the directions of the sun's tendrils. For each vertex the dot product between the vertex and the direction of each tendril is taken. The vertices that point more directly toward a tendril direction are pushed farther outward because their dot product with the generated tendril location will be really close to 1. The other vertices that are farther away are pushed less. This creates the pointed regions of the sun instead of uniformly expanding the sphere.

Finally, the object uses Unity's Transform.RotateAround function to continuously orbit around a specified point or just the world origin. The orbit axis, radius, and speed can be adjusted in Unity Inspector and I ended up going with a less realistic speed of orbit but I think it made it more fun to look at.

Clickable Earth:

The main system is a sphere that goes between a water-covered planet and a planet with raised continents which morphs triggered by clicking the mouse or a VR controller, while its color shifts from blue to green at the same time. The sphere itself is built with a cube sphere instead of a standard UV sphere, meaning it starts as a cube that gets subdivided and pushed outward into a rounded shape, which keeps the vertices spaced more evenly across the surface than a normal sphere would which helps during the continents raising section.

At startup, two full sets of vertex positions are generated from that same base shape. The first is a "water" set, which is just the plain sphere at its normal radius. Then the second "earth" set is generated. For the "earth" set each vertex's direction is run through Perlin noise, and any vertex whose noise value clears a set threshold gets pushed outward to form a small raised landmass. Vertices that don't clear the threshold just stay at the sphere's radius, so the land ends up scattered in patches instead of covering the whole sphere. I used a similar approach as the continuous deformation of the sun and moon but making it user controlled.

Each click nudges the morph value up or down by a small step between fully water (0) and fully land (1). Every frame, the script goes between the stored water and earth vertex depending on the that morph value and then just changes the mesh to reflect that. It also blends

the material's color between blue and green using the same value. I changed the input to check for both mouse clicks and vr triggers to make testing before going to The Roost easier.

In the tuning I ended up going with, the color change ends up being a lot more noticeable than the shape change, since the raised landmasses only push out a small amount compared to the sphere's overall size. I wanted to make the land mass heights small so it would be kind of similar to actual earth proportions but it just makes them barely noticeable.

Asteroid Field:

The main system is a procedural asteroid field that creates multiple randomly shaped asteroids and moves them slowly through the background. The asteroids are generated through code instead of using pre-made models, and each one has its own size, shape, movement direction, and rotation.

When the scene begins it creates a user-set number of asteroids and places them at random positions within a circular/spherical area around the object. Each asteroid starts as a cube that is converted into a sphere using the cube sphere utility as before. Perlin noise is then used to change the distance of each vertex from the center of the asteroid. Perlin noise helps make the bumps seem less out of place and makes the changes look smooth (similar to the sun). This gives the asteroids a lumpy and uneven shape instead of making them perfect spheres. Random values are also used for the asteroid's size, drift speed, and rotation speed so that they do not all look or move exactly the same but that can be changed.

During each frame, the asteroids slowly move in their randomly chosen directions and rotate around randomly chosen axes. The script also keeps track of how long each asteroid has existed. As it gets closer to the end of its lifetime, the vertices start collapsing toward the center of the asteroid. Each vertex has its own random threshold for when it starts collapsing, so the asteroid does not disappear all at once. Once the asteroid has fully crumbled, the script generates a new shape in the same position. I did this to make it seem more like the asteroids were breaking apart as they floated through space colliding with other space objects.

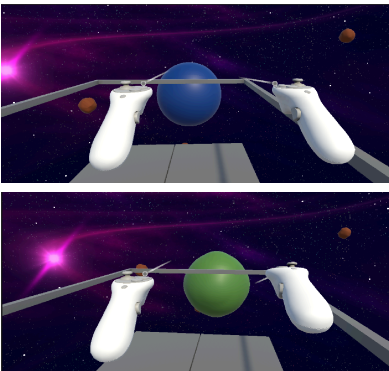
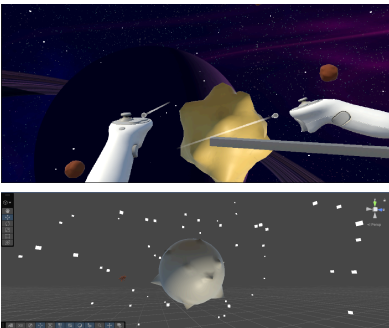
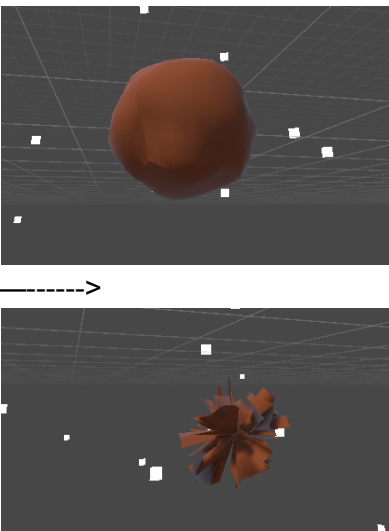
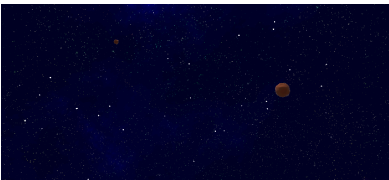
The asteroids also have a looping system so that when one drifts far enough away from the field, it is moved back to a random position within the original area. This keeps the background filled with asteroids without needing to constantly create and destroy new GameObjects.

Overall, almost everything about the asteroid can be customized in the Unity editor which I found really fun to mess around with. Seeing huge asteroids going really fast was quite amusing and made it so the user really gets the look they're hoping for.

CubeSphereUtility.cs / ProceduralMeshUtility.cs

These are some helper functions that are used in all 3 of the main deformation goals. Their functions are mentioned in the above script definitions. ^^^^

3. Evidence of requirements

Requirement	Where it is demonstrated	Screenshot
User Input	0:10 - 0:18; Screenshots 2 and 3. If you look really closely the green ball has a slightly different shape as well.	
Continuous Deformation	0:25 - 0:40 ; Screenshots 1 and 6	
Asteroid (my own choice)	0:05 - 0:50 (they're always in the background); Screenshots 7 and 8 show the asteroid breaking down and the vertices zeroing in on itself creates the breaking effect.	
I added white stars in the background as well as a custom skybox (SpaceBackground.cs)	The whole video	

5.)

The only asset I used was the skybox which I found here:

<https://assetstore.unity.com/packages/2d/textures-materials/sky/free-skyboxes-space-178953>

by Dogmatic

6.)

Tool	Used for	What was incorporated, changed, or rejected
Claude	Writing SpaceBackground.cs	Claude wrote the whole script (sky dome + stars), I placed it in the scene and kept it basically exactly the same
Claude	Debugging SunMoonCycle.cs	Claude found that a hardcoded position in Start() was teleporting the object off-screen and helped me switch the axis of rotation
Claude	Fixing input handling in ClickMorphFace.cs	I kept having issues getting the clicks on my mouse to work the same as the controllers and Claude helped me fix that
Claude	Adding the crumbling effect to AsteroidField.cs	I got caught up creating the asteroids and realized that there wasn't a deformation and Claude helped me create the breaking apart effect.
Claude	General troubleshooting	I ran into a ton of issues not knowing where to find certain tabs or why my code was acting weird and Claude helped a lot.

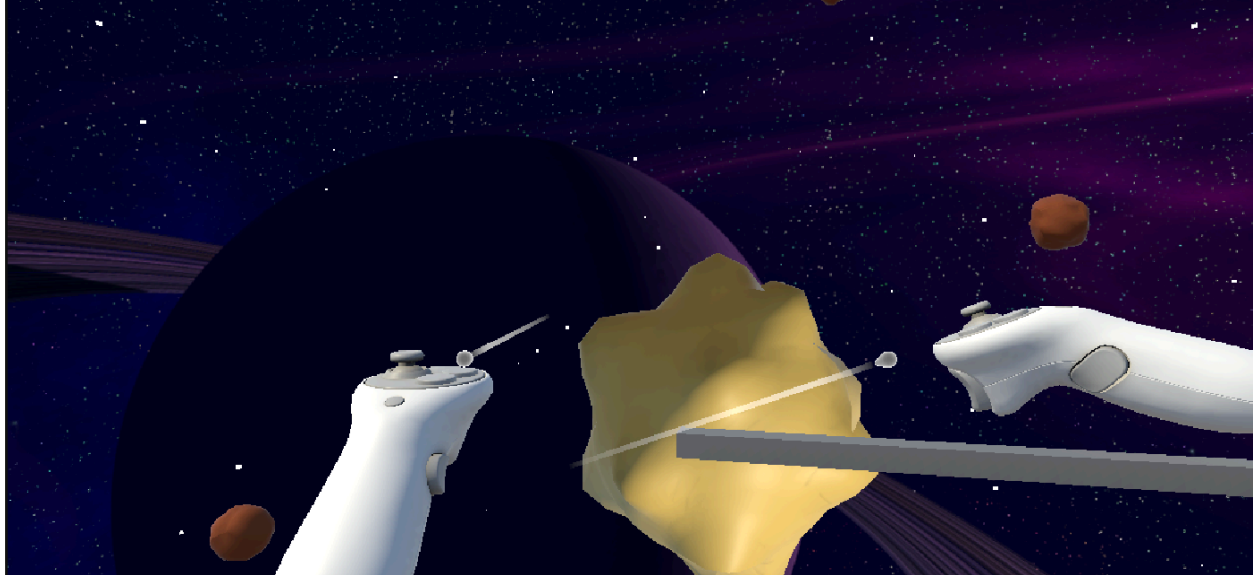
7.)

I found getting things to look the way I imagined in my head to be much more technically involved than I had imagined. I thought making the sun look like a kid's drawing with the spikes coming off of it to be a small task but I had to learn a lot to figure out how to get it to work and watch a decent number of youtube videos. I'd like to explore more on the asteroids because I still think it would be very cool if I could make chunks of the asteroids fall off in more natural shapes and linger around too. I had also imagined I could've got a way more realistic prehistoric and modern earth look to go between but it was much more difficult than just putting two images on a sphere.

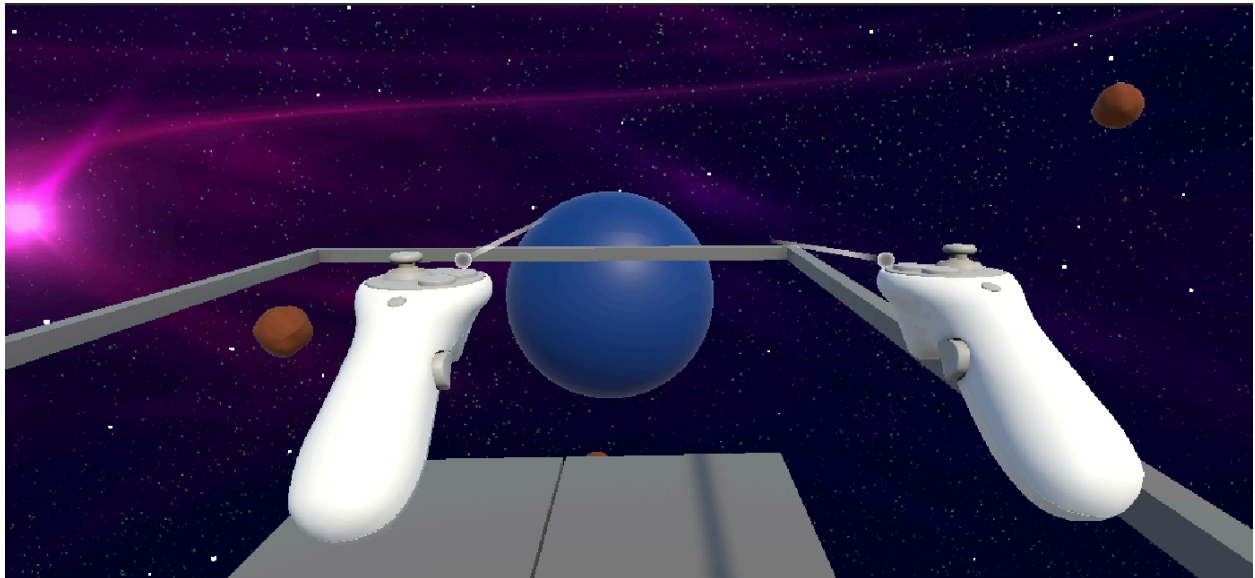
I learned a lot about vertices. Having to make two vertex pairs and shift through them was really cool but also challenging to wrap my head around at first. Also, generating the sun tendrils in a

procedural way and then learning how to get that to actually work despite the vertices being much different than I expected.

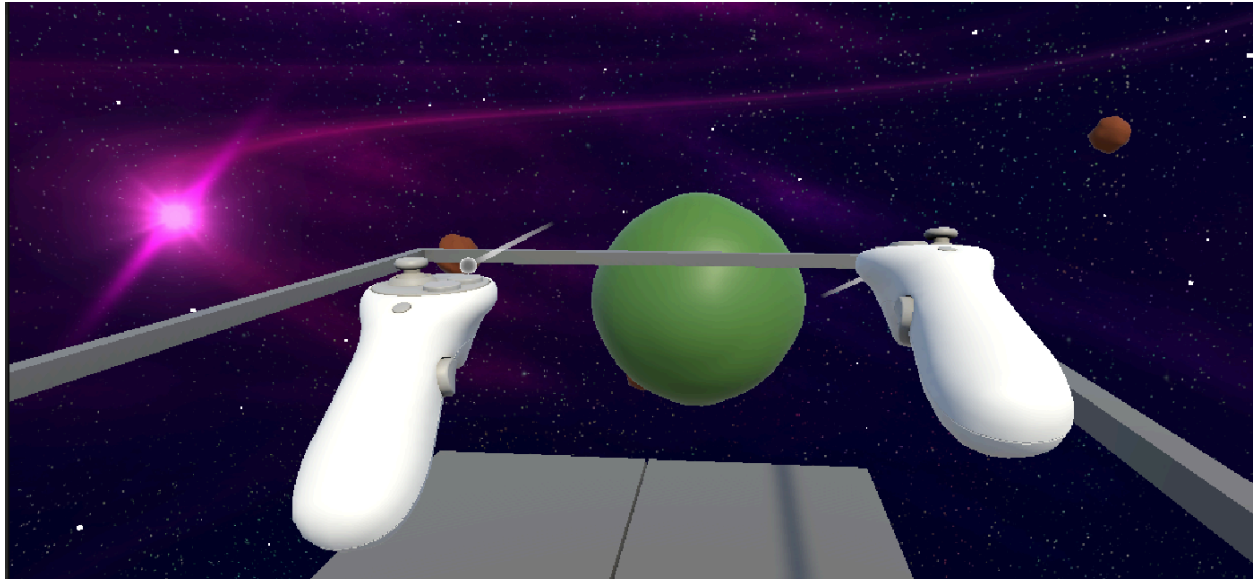
Screenshots: Screenshots
1 Sun deformation



2 Ancient earth before user input



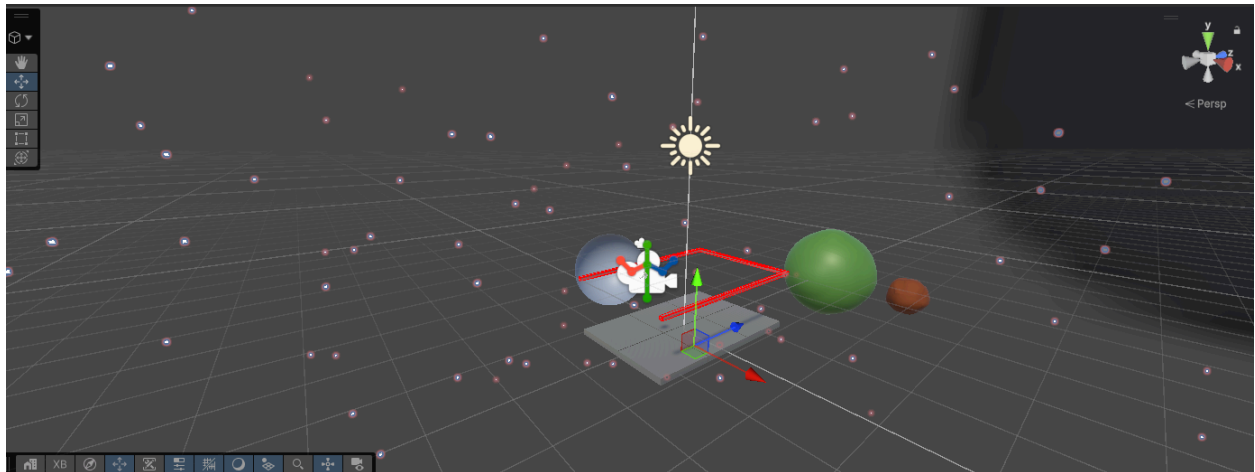
3 Modern Earth post user input



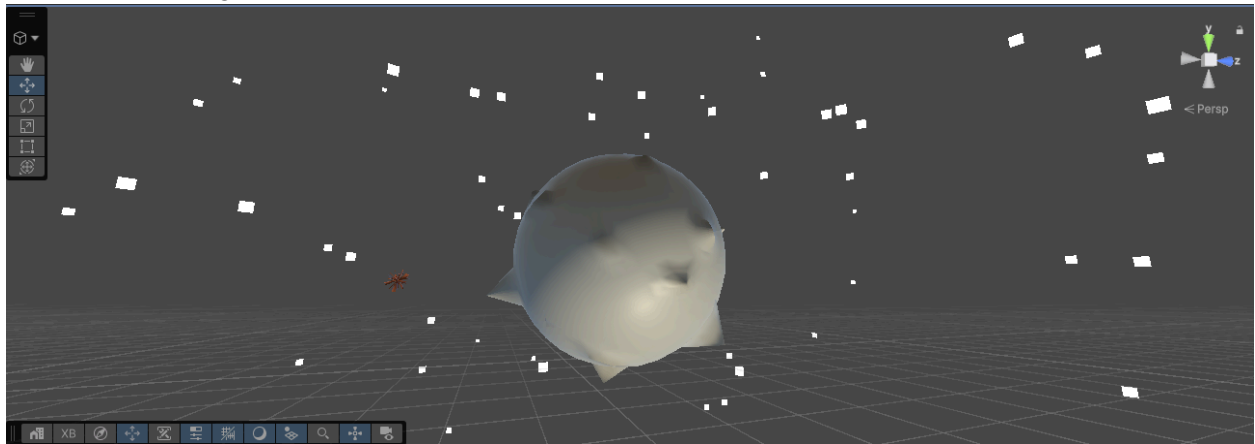
4 Cool space background



5 Moon deformation



6 In between stage deformation



7 & 8 Asteroid time deformation

